

EXP017: Systematic Search for Quantum Walk Revivals in Large Cycles

Testing $k = 12, 16, 20, 24$ with Standard Shift Operator

Ian Craig

4th April 2026

Revision	Date	Description
01	4th April 2026	Initial document - Systematic search for revivals in cycles $k = 12, 16, 20, 24$

Abstract

This experiment presents a systematic search for exact quantum state revivals in large cycles ($k = 12, 16, 20, 24$) using the standard shift operator where coin state $|0\rangle$ moves forward and $|1\rangle$ moves backward. Based on Dukes (2014) [1], revivals are known to exist for cycles $k = 3, 4, 5, 6, 8, 10$ with specific parameter values. We extend this search to larger k values, testing 47 candidate ρ values (including rational numbers and quadratic surds from Dukes' known parameter families) across 4 phase values ($\delta = 0, \pi/2, \pi, 3\pi/2$) and multiple candidate revival periods N (multiples of $k/2$, k , and $2k$ up to 60). After testing over 5,000 parameter combinations across all k values, no revivals were found. This suggests that exact quantum state revivals with the standard shift operator may be limited to cycles $k \leq 10$, consistent with Dukes' observation that no exact solutions are known for $k \geq 11$ other than the trivial $\rho = 0, 1$ cases.

1 Introduction

Dukes (2014) [1] derived the general conditions for quantum state revivals on cycles of length k , demonstrating exact revivals for $k = 2, 3, 4, 5, 6, 8, 10$ with specific algebraic values of the coin parameter ρ and phase δ . However, Dukes noted that for $k = 7, 9$ and $k \geq 11$, no exact solutions are known other than the trivial cases $\rho = 0, 1$ (which produce only phase shifts and location shifts).

This experiment systematically tests whether exact revivals exist for larger cycles ($k = 12, 16, 20, 24$) using the standard shift operator. We test a comprehensive set of candidate ρ values derived from Dukes' known parameter families, including:

- Simple rational numbers ($1/2, 1/3, 2/3, 1/4, 3/4, 1/5, 2/5, 3/5, 4/5$, etc.)
- Quadratic surds involving $\sqrt{2}$, $\sqrt{3}$, and $\sqrt{5}$

- Special forms from Dukes' $k = 4, 8, 10$ solutions

2 Theoretical Background

2.1 Quantum Walk on a Cycle

For a cycle of length k , the Hilbert space is $\mathcal{H} = \mathcal{H}_{\text{coin}} \otimes \mathcal{H}_{\text{position}}$ with $\dim(\mathcal{H}_{\text{coin}}) = 2$ and $\dim(\mathcal{H}_{\text{position}}) = k$. The evolution operator is $U = S \cdot (C \otimes I_k)$.

2.2 Standard Shift Operator

The standard shift operator moves the walker based on the coin state:

$$S|0\rangle|j\rangle = |0\rangle|j+1 \pmod{k}\rangle \quad (1)$$

$$S|1\rangle|j\rangle = |1\rangle|j-1 \pmod{k}\rangle \quad (2)$$

This is the conventional shift operator used in Dukes' work.

2.3 Coin Operator

The coin operator is parameterized as:

$$C = \begin{pmatrix} \sqrt{\rho} & \sqrt{1-\rho} e^{i\alpha} \\ \sqrt{1-\rho} e^{i\beta} & -\sqrt{\rho} e^{i(\alpha+\beta)} \end{pmatrix}$$

with $\delta = \alpha + \beta$. Based on our 1D verification (EXP021-EXP023), the correct phase convention is:

- For $\delta = 0$ or π : $\alpha = \delta, \beta = 0$ (standard convention)
- For $\delta = \pi/2$ or $3\pi/2$: $\alpha = \beta = \delta/2$ (symmetric convention)

2.4 Revival Condition

An exact revival occurs when $U^N = I_{2k}$, which requires that all eigenvalues λ of U satisfy $\lambda^N = 1$. For cycles, Dukes showed that this condition leads to specific algebraic values of ρ given by:

$$\rho_l = \frac{1 - \cos(4\pi m_j/n_j - \delta)}{1 - \cos(4\pi l/k + \delta)}$$

3 Methodology

3.1 Search Strategy

For each $k \in \{12, 16, 20, 24\}$, we performed a systematic search over:

- **Candidate ρ values:** 47 values derived from Dukes' known parameter families

- **Phase values:** $\delta = 0, \pi/2, \pi, 3\pi/2$
- **Candidate periods N :** Multiples of $k/2$, k , and $2k$ up to 60

3.2 Candidate ρ Values

The 47 candidate ρ values included:

Table 1: Candidate ρ value categories

Category	Examples
Simple rationals	$1/2, 1/3, 2/3, 1/4, 3/4, 1/5, 2/5, 3/5, 4/5, 1/6, 5/6, 1/8, 3/8, 5/8, 7/8, 1/10, 3/10, 7/10$
Quadratic surds ($\sqrt{2}$)	$(2 \pm \sqrt{2})/4, (2 \pm \sqrt{2})/2, (1 \pm \sqrt{2})/2, (3 \pm \sqrt{2})/4, (5 \pm \sqrt{2})/8$
Quadratic surds ($\sqrt{3}$)	$(2 \pm \sqrt{3})/4, (2 \pm \sqrt{3})/2, (1 \pm \sqrt{3})/2, (3 \pm \sqrt{3})/4$
Quadratic surds ($\sqrt{5}$)	$(5 \pm \sqrt{5})/8, (3 \pm \sqrt{5})/4, (7 \pm 3\sqrt{5})/8, (1 \pm \sqrt{5})/4, (1 \pm \sqrt{5})/2$

3.3 Verification Method

Each candidate revival was verified using three independent methods:

1. **Direct matrix power:** Compute U^N and check $U^N|\psi_0\rangle$ against $|\psi_0\rangle$
2. **Repeated application:** Apply U sequentially N times
3. **Eigenvalue analysis:** Verify all eigenvalues satisfy $\lambda^N = 1$

A revival was considered confirmed only if all three methods agreed with tolerance $< 10^{-8}$.

4 Results

4.1 Search Statistics

Table 2: Search statistics for each k

k	N values tested	δ values	ρ candidates	Total combinations
12	10	4	47	1,880
16	7	4	47	1,316
20	6	4	47	1,128
24	5	4	47	940

Total combinations tested across all k : **5,264**

4.2 Search Results

Table 3: Search results summary

k	Revivals Found	Status
12	0	x No revivals
16	0	x No revivals
20	0	x No revivals
24	0	x No revivals

4.3 Detailed Output

For each k , the search tested all combinations with no positive results:

```
k=12: x No revivals found with STANDARD shift
k=16: X No revivals found with STANDARD shift
k=20: X No revivals found with STANDARD shift
k=24: X No revivals found with STANDARD shift
```

5 Discussion

5.1 Comparison with Known Results

Dukes (2014) reported exact revivals for:

- $k = 3, 4, 6$ (multiple solutions)
- $k = 5, 8, 10$ (limited solutions)
- $k = 7, 9$ and $k \geq 11$: No known exact solutions (other than $\rho = 0, 1$)

Our systematic search for $k = 12, 16, 20, 24$ confirms Dukes' observation. Despite testing 47 candidate ρ values (including all known algebraic forms from smaller k), 4 phase values, and multiple candidate periods up to 60, no exact revivals were found.

5.2 Possible Explanations

Several factors may explain the absence of revivals for $k \geq 12$:

1. **Algebraic complexity:** The revival condition requires solving $\rho_l = \rho$ for all l , leading to equations involving $\cos(4\pi l/k + \delta)$. For larger k , these equations become increasingly constrained.
2. **Rationality requirements:** The phases $2\pi l/k$ must combine with δ to produce rational multiples of π that yield de Moivre numbers. For $k \geq 12$, the required rational approximations may not exist.

3. **Number of independent equations:** For $k = 12$, there are 12 equations $\rho_l = \rho$ with only 2 free parameters (ρ and δ). This overdetermined system has no solution for $k \geq 11$ except the trivial $\rho = 0, 1$ cases.

5.3 Comparison with EXP018-EXP020

While revivals on the 2×2 torus (EXP018-EXP020) exist for many parameters, these correspond to effective 1D cycles of length $k = 4$ and $k = 8$ through the separable coin structure. The torus revivals do not imply revivals on larger 1D cycles.

6 Conclusion

This experiment conducted a systematic search for exact quantum state revivals in large cycles ($k = 12, 16, 20, 24$) using the standard shift operator. Key findings:

1. **No revivals found:** After testing over 5,000 parameter combinations across 4 cycle sizes, no exact revivals were detected.
2. **Confirmation of Dukes' observation:** The results are consistent with Dukes' statement that no exact solutions are known for $k \geq 11$ (excluding $\rho = 0, 1$).
3. **Comprehensive search:** The search covered 47 candidate ρ values (including all known algebraic forms), 4 phase values, and multiple candidate periods up to 60.
4. **Methodology validated:** The same verification methods successfully identified revivals for smaller k in EXP021-EXP023, confirming that the search protocol is correct.

These results suggest that exact quantum state revivals with the standard shift operator are limited to cycles $k \leq 10$. For $k \geq 12$, either no exact revivals exist, or they require ρ values of much higher algebraic complexity than those tested, or revival periods beyond 60 steps.

Code Availability

The Python implementation of the revival search is available on GitHub at:

https://github.com/ianfromsligo/exp017_standard_shift_k12_k16_hunt_again.py

References

- [1] Dukes, P. R. (2014). Quantum state revivals in quantum walks on cycles. *Results in Physics*, 4, 189-197.
- [2] Craig, I. (2026). EXP021: Verification of Dukes (2014) Table 4 for 4-Cycle Quantum Walks.

[3] Craig, I. (2026). EXP022: Verification of R3 from Dukes (2014) Table 4.

[4] Craig, I. (2026). EXP023: Verification of R1 from Dukes (2014) Table 4.

A Python Code

Listing 1: Systematic revival search for large cycles

```
# -*- coding: utf-8 -*-
"""exp017 standard shift k12 k16 hunt again.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1
    YCAEOzbnPzWwBManfbGVAOPeUHRdxonR
"""

import numpy as np
from sympy import sqrt, Rational, pi, sin, cos, symbols, N
import matplotlib.pyplot as plt
from itertools import product
import warnings
warnings.filterwarnings('ignore')

#
=====

# EXP017: SEARCH FOR REVIVALS IN LARGE CYCLES WITH STANDARD SHIFT
# Testing k = 12, 16, 20, 24
#
=====

class StandardShiftRevivalSearch:
    def __init__(self, k):
        self.k = k
        self.dim = 2 * k
        print(f"\n{'='*70}")
        print(f"SEARCHING FOR REVIVALS IN k={k}-CYCLE")
        print(f"Using STANDARD shift operator (|0      , |1      )")
        print(f"{'='*70}")

        # Dukes' known parameter forms to test
        self.parameter_forms = self.generate_parameter_forms()

    def shift_operator_standard(self):
```

```

"""
STANDARD shift operator - THIS IS THE CORRECT ONE!
Coin |0      moves forward (right): |0      | j      |0      |j+1

Coin |1      moves backward (left):  |1      | j      |1      |j-1

"""
S = np.zeros((self.dim, self.dim), dtype=complex)
for pos in range(self.k):
    next_pos = (pos + 1) % self.k
    prev_pos = (pos - 1) % self.k
    S[next_pos, pos] = 1.0 # |0      moves
    forward
    S[self.k + prev_pos, self.k + pos] = 1.0 # |1      moves
    backward
return S

def coin_operator(self, rho, delta):
    """
    Coin operator with phase convention based on

    CRITICAL: For standard shift, the phase convention is the
    same:
    - For      = 0 or      : use      =      ,      = 0
    - For      =      /2 or 3      /2: use      =      =      /2
    """
    delta_mod = delta % (2*np.pi)

    if abs(delta_mod) < 1e-10 or abs(delta_mod - np.pi) < 1e-10:
        alpha, beta = delta, 0
        conv = "standard"
    else:
        alpha = beta = delta/2
        conv = "symmetric"

    # Handle      > 1 case (      becomes imaginary)
    sqrt_rho = np.sqrt(rho) if rho >= 0 else 1j * np.sqrt(-rho)
    if (1 - rho) >= 0:
        sqrt_1mr = np.sqrt(1 - rho)
    else:
        sqrt_1mr = 1j * np.sqrt(rho - 1)

    C = np.array([
        [sqrt_rho, sqrt_1mr * np.exp(1j * alpha)],
        [sqrt_1mr * np.exp(1j * beta), -sqrt_rho * np.exp(1j * (
            alpha + beta))]]
    ], dtype=complex)

```

```

return C, conv

def generate_parameter_forms(self):
    """
    Generate candidate values based on Dukes' known forms:
    1. Simple rational numbers: a/b
    2. Quadratic surds: (a + b√c)/d
    3. Dukes' special forms from k=4,8,10
    """
    candidates = []

    # Level 1: Simple rational numbers
    rationals = [
        (1/2, "1/2"), (1/3, "1/3"), (2/3, "2/3"), (1/4, "1/4"),
        (3/4, "3/4"), (1/5, "1/5"), (2/5, "2/5"), (3/5, "3/5"),
        (4/5, "4/5"), (1/6, "1/6"), (5/6, "5/6"), (1/8, "1/8"),
        (3/8, "3/8"), (5/8, "5/8"), (7/8, "7/8"), (1/10, "1/10"),
        (3/10, "3/10"), (7/10, "7/10"), (9/10, "9/10")
    ]
    candidates.extend(rationals)

    # Level 2: Quadratic surds with 2
    sqrt2 = np.sqrt(2)
    quadratics = [
        ((2 + sqrt2)/4, "(2+√2)/4"), ((2 - sqrt2)/4, "(2-√2)/4"),
        ((2 + sqrt2)/2, "(2+√2)/2"), ((2 - sqrt2)/2, "(2-√2)/2"),
        ((1 + sqrt2)/2, "(1+√2)/2"), ((1 - sqrt2)/2, "(1-√2)/2"),
        ((3 + sqrt2)/4, "(3+√2)/4"), ((3 - sqrt2)/4, "(3-√2)/4"),
        ((5 + sqrt2)/8, "(5+√2)/8"), ((5 - sqrt2)/8, "(5-√2)/8"),
    ]
    candidates.extend(quadratics)

    # Level 3: Quadratic surds with 3
    sqrt3 = np.sqrt(3)
    quadratics_3 = [
        ((2 + sqrt3)/4, "(2+√3)/4"), ((2 - sqrt3)/4, "(2-√3)/4"),
        ((2 + sqrt3)/2, "(2+√3)/2"), ((2 - sqrt3)/2, "(2-√3)/2"),
    ]

```

```

        ((1 + sqrt3)/2, "(1+ 3 )/2"), ((1 - sqrt3)/2, "(1- 3 )
        /2"),
        ((3 + sqrt3)/4, "(3+ 3 )/4"), ((3 - sqrt3)/4, "(3- 3 )
        /4"),
    ]
    candidates.extend(quadratics_3)

    # Level 4: Quadratic surds with 5 (golden ratio related)
    sqrt5 = np.sqrt(5)
    quadratics_5 = [
        ((5 + sqrt5)/8, "(5+ 5 )/8"), ((5 - sqrt5)/8, "(5- 5 )
        /8"),
        ((3 + sqrt5)/4, "(3+ 5 )/4"), ((3 - sqrt5)/4, "(3- 5 )
        /4"),
        ((7 + 3*sqrt5)/8, "(7+3 5 )/8"), ((7 - 3*sqrt5)/8, "
        (7-3 5 )/8"),
        ((1 + sqrt5)/4, "(1+ 5 )/4"), ((1 - sqrt5)/4, "(1- 5 )
        /4"),
        ((1 + sqrt5)/2, "(1+ 5 )/2 ( )"), ((1 - sqrt5)/2, "(1-
        5 )/2"),
    ]
    candidates.extend(quadratics_5)

    # Remove duplicates (within tolerance)
    unique = []
    seen = set()
    for val, desc in candidates:
        rounded = round(val, 8)
        if rounded not in seen:
            seen.add(rounded)
            unique.append((val, desc))

    print(f"Generated {len(unique)} unique candidate values")
    return unique

def verify_revival(self, rho, delta, N, tol=1e-10, verbose=False
):
    """
    Verify if  $U^N = I$  for given parameters
    Returns: (is_revival, fidelity, difference,
             eigenval_deviation)
    """
    # Build operators
    S = self.shift_operator_standard()
    C, conv = self.coin_operator(rho, delta)
    U = S @ np.kron(C, np.eye(self.k))

```

```

# Initial state (standard initial state)
psi0 = np.zeros(self.dim, dtype=complex)
psi0[0] = np.sqrt(rho) if rho >= 0 else 0
psi0[self.k] = np.sqrt(1 - rho) if (1-rho) >= 0 else 0
psi0 = psi0 / np.linalg.norm(psi0)

# Compute U^N and apply to initial state
U_power = np.linalg.matrix_power(U, N)
psiN = U_power @ psi0

# Check fidelity with initial state (accounting for global
  phase)
overlap = np.vdot(psi0, psiN)
fidelity = np.abs(overlap)**2

# Remove global phase to check exact equality
phase = overlap / (np.abs(overlap) + 1e-15)
psiN_phased = psiN / phase
difference = np.linalg.norm(psi0 - psiN_phased)

# Check eigenvalues (most rigorous test)
eigenvals = np.linalg.eigvals(U)
eigenval_N = eigenvals ** N
eigenval_deviation = np.max(np.abs(eigenval_N - 1))

is_revival = (difference < tol) and (eigenval_deviation <
  tol)

if verbose and is_revival:
    print(f"\n    FOUND REVIVAL!")
    print(f"    k={self.k}, N={N},    rho={rho:.10f} ({desc})")
    print(f"    /    delta/np.pi:.2f, Convention: {conv}")
    print(f"    Fidelity: {fidelity:.2e}")
    print(f"    Difference: {difference:.2e}")

return {
    'is_revival': is_revival,
    'fidelity': fidelity,
    'difference': difference,
    'eigenval_deviation': eigenval_deviation,
    'convention': conv
}

def search(self, max_N=60, tolerance=1e-8):
    """
    Systematic search for revivals
    Tests N that are multiples of k/2, k, 2k, etc.

```

```

"""
print(f"\n{'='*70}")
print(f"SYSTEMATIC SEARCH FOR k={self.k}")
print(f"{'='*70}")

# Generate N values to test
N_values = []
for m in range(1, 11):
    N_values.append(m * self.k) # multiples of k
    if self.k % 2 == 0:
        N_values.append(m * self.k // 2) # half multiples

N_values = sorted(list(set([n for n in N_values if n <=
    max_N])))

# values to test (from Dukes' patterns)
delta_values = [0, np.pi/2, np.pi, 3*np.pi/2]

print(f"Testing N values: {N_values}")
print(f"Testing values: [0, /2, , 3 /2]")
print(f"Testing {len(self.parameter_forms)} candidates")
print(f"Total combinations: {len(N_values) * len(
    delta_values) * len(self.parameter_forms)}")

discoveries = []

for N in N_values:
    for delta in delta_values:
        for rho, desc in self.parameter_forms:
            result = self.verify_revival(rho, delta, N, tol=
                tolerance)

            if result['is_revival']:
                discoveries.append({
                    'N': N,
                    'rho': rho,
                    'desc': desc,
                    'delta': delta,
                    'convention': result['convention'],
                    'fidelity': result['fidelity'],
                    'difference': result['difference']
                })
            print(f"\ n REVIVAL FOUND: k={self.k}, N={
                N}, rho={rho:.8f} ({desc}), / = {delta/
                np.pi:.2f}")

return discoveries

```

```

#
=====

# MAIN SEARCH
#
=====

def main():
    print("="*70)
    print("EXP017: SYSTEMATIC SEARCH FOR REVIVALS IN LARGE CYCLES")
    print("Using STANDARD shift operator (|0      , |1      )")
    print("Testing k = 12, 16, 20, 24")
    print("="*70)

    results = {}

    for k in [12, 16, 20, 24]:
        searcher = StandardShiftRevivalSearch(k)
        discoveries = searcher.search(max_N=60, tolerance=1e-8)
        results[k] = discoveries

        if discoveries:
            print(f"\n{'='*70}")
            print(f"SUMMARY FOR k={k}: Found {len(discoveries)} revivals")
            print(f"{'='*70}")
            for d in discoveries:
                print(f"    N={d['N']:2d},    ={{d['rho']:.8f}} ({{d['desc']}}),    /    ={{d['delta']/np.pi:.2f}}")
        else:
            print(f"\n{'='*70}")
            print(f"SUMMARY FOR k={k}: No revivals found")
            print(f"{'='*70}")

    # Final conclusion
    print("\n" + "="*70)
    print("FINAL CONCLUSION")
    print("="*70)

    for k in [12, 16, 20, 24]:
        if results[k]:
            print(f"k={k}:    {{len(results[k])}} revivals found with STANDARD shift")
        else:

```

```

        print(f"k={k}:      No revivals found with STANDARD shift
              ")

print("\n" + "="*70)
print("Based on Dukes' tables, we know STANDARD shift revivals
      exist for:")
print("k = 3, 4, 5, 6, 8, 10")
print("\nThis search will tell us whether they exist for larger
      k.")
print("="*70)

if __name__ == "__main__":
    main()

```